centriMembSolver — Theory, Implementation and Usage Guide

A combined user and developer manual for rotating membrane simulations

Vítor Geraldes
Department of Chemical Engineering
Instituto Superior Técnico — Universidade de Lisboa
vitor.geraldes@tecnico.ulisboa.pt

May 10, 2026

License and trademarks

Copyright. © 2026 Vítor Geraldes.

Software. `centriMembSolver` is distributed under the GNU General Public License version 3 (GPLv3) or later. See the `LICENSE` file distributed with the source code. The software is provided "as is", WITHOUT WARRANTY OF ANY KIND, express or implied, including but not limited to the warranties of merchantability, fitness for a particular purpose and noninfringement; see the GPL for details.

Documentation. This manual is licensed under the Creative Commons Attribution 4.0 International License (CC BY 4.0), unless stated otherwise.

Trademarks. OpenFOAM® is a registered trademark of OpenCFD Limited (ESI Group). Linux® is a registered trademark of Linus Torvalds. ParaView is a trademark of Kitware Inc. This work is not affiliated with, endorsed by, or sponsored by OpenCFD Limited, the Linux Foundation, or Kitware Inc.

How to cite

`centriMembSolver` is distributed as an open-source platform for the community of rotating-membrane simulations. Users who publish work based on the solver are encouraged to cite the versioned software release directly:

> Geraldes, V., *centriMembSolver: a finite-volume solver for rotating membrane systems built on OpenFOAM® technology*, version 1.1.0, Zenodo, **[DOI to be inserted upon Zenodo deposit]**, 2026.

A BibTeX entry is provided in the source repository (`github.com/vitormgeraldes/centriMembSolver`).

Application example. A first quantitative application of `centriMembSolver` to the prediction of concentration polarisation in Ortho-Centrifugal Membrane Filters is described in Geraldes *et al.* (*Predicting concentration polarization in Ortho-Centrifugal Membrane Filters for NF/RO membrane screening*, submitted to *Journal of Membrane Science*). That work documents a buoyancy-driven mass-transfer correlation derived from a direct numerical simulation dataset produced with this solver and is the appropriate reference for users adopting the same physical framework.

Methodological extensions. Users developing new boundary conditions, transport closures, multi-physics couplings, or new application studies are encouraged to publish their own descriptions and to reference the `centriMembSolver` software DOI as the implementation platform. Pull requests with new tutorials, BC classes or transport models are welcome via the source repository.

Abstract

`centriMembSolver` is a finite–volume solver for coupled solvent and solute transport through rotating membranes. It extends `buoyantBoussinesqPimpleFoam` to include solute transport and semi-permeable membrane boundary conditions, concentration-dependent viscosity, and an explicit treatment of the Coriolis force consistent with the Boussinesq buoyancy formulation under centrifugal acceleration. This guide documents the underlying theory, the implementation of membrane flux models, and the overall solver architecture. A worked example, *rotatingSlit*, is included to illustrate case setup and post-processing. The document is intended both for end-users running membrane simulations and for developers wishing to extend the solver to new flux laws or multi-physics couplings.

Contents

How to cite 2

Abstract 3

Notation 5

1 Introduction 7

Quick start . . . 7

2 Theory and Governing Equations 8

2.1 Hydrodynamics . . . 8

2.1.1 Modified pressure, hydrostatic potential and Boussinesq buoyancy . . . 8

2.1.2 Turbulence modelling . . . 11

2.1.3 Concentration-dependent viscosity . . . 12

2.2 Solute and tracer transport . . . 12

2.3 Coriolis treatment in PIMPLE . . . 12

3 Membrane Boundary Conditions 12

3.1 Membrane physics . . . 13

3.2 Solvent flux boundary (`U`) . . . 15

3.3 Solute flux boundary (`CA`) . . . 16

3.4 Tracer flux boundary (`CT`) . . . 16

3.5 Membrane patch dictionaries . . . 17

4 Solver structure and implementation 17

4.1 Overall time loop and custom PIMPLE algorithm . . . 18

4.2 Field creation, membrane diagnostics and transport properties . . . 20

4.3 Momentum and pressure equations . . . 20

4.4 Membrane boundary conditions and the Γ diagnostic . . . 21

4.5 Post-processing: pressure reconstruction, conservation and diagnostics . . . 21

5 Case studies with `centriMembSolver` 22

5.1 `rotatingSlit` . . . 22

6 Practical guidance 24

7 References 25

Notation

The following table maps the mathematical symbols used throughout this manual to the corresponding OpenFOAM field names and dictionary keys.

Symbol	Quantity	Field / dict key	Units
$\boldsymbol{U}$	velocity field (rotating frame)	`U`	$\mathrm{m\,s^{-1}}$
p_{rgh}	modified (dynamic) pressure	`p_rgh`	Pa
gh	combined gravitational + centrifugal hydrostatic potential per unit mass	`gh`	$\mathrm{m^2\,s^{-2}}$
ρ_k	dimensionless density ratio ρ/ρ_0	`rhok`	–
$\mathbf{r}$	position vector relative to the rotation axis	—	m
$\mathbf{r}_\perp$	component of $\mathbf{r}$ perpendicular to the rotation axis	—	m
$\boldsymbol{\Omega}$	angular velocity vector (along rotation axis)	`SRFProperties` (`rpm`, `axis`)	$\mathrm{rad\,s^{-1}}$
Ω	angular speed, $\Omega = \lvert\boldsymbol{\Omega}\rvert$	—	$\mathrm{rad\,s^{-1}}$
r_m	membrane orbit radius (for centrifugal NF/RO cartridges)	—	m
a_m	centrifugal acceleration at the membrane, $a_m = \Omega^2 r_m$	—	$\mathrm{m\,s^{-2}}$
C_A	primary solute concentration	`CA`	$\mathrm{kg\,m^{-3}}$
$D_A(C_A)$	primary solute diffusivity (concentration–dependent)	`speciesData` (`DAB0..4`)	$\mathrm{m^2\,s^{-1}}$
$\nu(C_A)$	kinematic viscosity (concentration–dependent)	`speciesData` (`nu0..2`)	$\mathrm{m^2\,s^{-1}}$
β	solutal contraction coefficient $(1/\rho_0)(\partial\rho/\partial C_A)$	`speciesData` (`beta`)	$(\mathrm{kg\,m^{-3}})^{-1}$
ρ_0	reference density (water at T_{ref})	`speciesData` (`rho0`)	$\mathrm{kg\,m^{-3}}$
$\pi(C_A)$	osmotic pressure	via `virialCoeffs`	Pa
C_T	tracer (passive secondary solute) concentration	`CT`	$\mathrm{kg\,m^{-3}}$
D_T	tracer diffusivity (constant)	`transportProperties` (`DT0`)	$\mathrm{m^2\,s^{-1}}$
J_v	solvent (water) flux through membrane	patch field `Jv`	$\mathrm{m\,s^{-1}}$
J_s	solute mass flux through membrane	patch field `Js`	$\mathrm{kg\,(m^2\,s)^{-1}}$
$J_{s,T}$	tracer mass flux through membrane	patch field `JsT`	$\mathrm{kg\,(m^2\,s)^{-1}}$
C_w, C_p	solute concentrations at the membrane wall (feed side) and permeate side	internal / `CAp`	$\mathrm{kg\,m^{-3}}$
C_f, C_b	feed-side and bulk reference concentrations	dict (`CAb`)	$\mathrm{kg\,m^{-3}}$
A_h	hydraulic permeability of membrane	`Ah`	$\mathrm{m\,(Pa\,s)^{-1}}$
σ	solute reflection coefficient (Spiegler–Kedem)	`sigma`	–
σ_T	tracer reflection coefficient (default 1)	`sigmaT`	–
B	solute permeability coefficient (Kedem–Katchalsky)	`B`	$\mathrm{m\,s^{-1}}$
B_T	tracer permeability coefficient	`BT`	$\mathrm{m\,s^{-1}}$
R_{int}, R_{obs}	intrinsic and observed solute rejection	`Rint`, `Robs`	–
$R_{T,\mathrm{int}}$, $R_{T,\mathrm{obs}}$	intrinsic and observed tracer rejection	`RTint`, `RTobs`	–
Γ	solute concentration polarisation $(C_w - C_b)/C_b$	volume field `Gama`	–
Γ_T	tracer concentration polarisation	volume field `GamaT`	–

1 Introduction

Rotating membrane systems combine fluid flow, solute transport and selective separation in a centrifugal field. Applications range from ultra–scale–down (USD) process development, where small membrane coupons are rotated at high speed to emulate industrial centrifugation, to high-throughput screening of nanofiltration and reverse osmosis (NF/RO) membranes in centrifugal filter cartridges. In these systems a pressure gradient drives solvent through a semi–permeable barrier while a concentration difference induces osmotic counterflow. Rotation adds centrifugal and Coriolis effects that modify the hydrodynamics and solute distribution.

`centriMembSolver` was developed to provide an open and customisable platform for simulating such coupled phenomena. Built on OpenFOAM® v2506, the solver solves the incompressible Navier–Stokes equations with source terms for rotation, a scalar transport equation for solute concentration and custom patch fields to model membrane fluxes. The Coriolis acceleration is treated as an explicit source term in the momentum equation, refreshed at every PIMPLE outer iteration; this is sufficient under the small CFL-bounded time steps characteristic of rotating membrane DNS. In addition, the membrane boundary conditions implement well established flux laws—Spiegler–Kedem, constant flux and target average flux for the solvent, and intrinsic rejection, solute permeability or observed rejection for the solute. Diagnostic quantities such as observed rejection and concentration polarisation are computed automatically. As of the 2026 release, the solver has been extended to simulate a primary solute A and a passive tracer T with separate diffusivities and membrane flux models, enabling tracer-based validation experiments.

The following sections present the governing equations (including viscous and osmotic effects), derive the robust Coriolis algorithm, detail the boundary conditions, describe the solver structure and illustrate usage through a case study. Developer notes at the end provide guidance on extending the code and best practice for stable simulations.

An example application of the solver, including validation against experimental data, is summarised on page 22 and described in detail in the work cited under *How to cite*.

Quick start

For users who want to run a case immediately, the following commands compile the libraries and the solver and execute the bundled two-dimensional tutorial `rotatingSlit`. `<repo>` denotes the root of the cloned source tree, and an active OpenFOAM environment (`source` of `etc/bashrc`) is assumed.

Serial (single-process) workflow.

1. Build the membrane boundary-condition and transport libraries, then the solver:

```
cd <repo>/src     && wmake all libso
cd <repo>/solver  && wmake
```

2. Run the bundled tutorial:

```
cd <repo>/tutorials/rotatingSlit
blockMesh
centriMembSolver
```

3. Visualise the results in ParaView (requires the `paraFoam` reader shipped with OpenFOAM):

```
paraFoam
```

Parallel (MPI) workflow. For three-dimensional cases the solver runs under MPI in the standard OpenFOAM idiom:

```
decomposePar                                    # split mesh+fields
mpirun -np <N> centriMembSolver -parallel       # run on N processes
reconstructPar                                  # merge fields for paraFoam
```

The number of processes `<N>` is set by `numberOfSubdomains` in `system/decomposeParDict`; standard `simple` or `scotch` decomposition methods are supported.

Section 5 walks through the bundled tutorial file by file and explains the diagnostic outputs.

2 Theory and Governing Equations

2.1 Hydrodynamics

In a non-inertial reference frame rotating with angular velocity $\mathbf{\Omega}$, the incompressible Navier–Stokes equations read

$$\nabla \cdot \mathbf{U} = 0, \qquad \rho \frac{\partial \mathbf{U}}{\partial t} + \rho \nabla \cdot (\mathbf{U}\mathbf{U}) = -\nabla p + \nabla \cdot (\mu_{\text{eff}} \nabla \mathbf{U}) + \rho \, \mathbf{f}_{\text{rot}}, \tag{1}$$

where $\mathbf{U}$ is the velocity, p the kinematic pressure ($p = p_{\text{phys}}/\rho$), and the effective viscosity $\mu_{\text{eff}} = \rho(\nu + \nu_t)$ includes molecular and—if selected—turbulent viscosity.

The rotational body force $\mathbf{f}_{\text{rot}}$ contains the two pseudo-forces appearing in a uniformly rotating frame, the centrifugal and Coriolis terms,

$$\mathbf{f}_{\text{rot}} = -\,\mathbf{\Omega} \times (\mathbf{\Omega} \times \mathbf{r}) - 2\,\mathbf{\Omega} \times \mathbf{U}, \tag{2}$$

with $\mathbf{r}$ the position vector relative to the rotation axis. The first term is conservative and points radially outward, $-\mathbf{\Omega} \times (\mathbf{\Omega} \times \mathbf{r}) = +\Omega^2 \, \mathbf{r}_\perp$; the second is non-conservative. Pressure–velocity coupling is handled by the PIMPLE algorithm (Pressure-Implicit Method for Pressure-Linked Equations, the standard OpenFOAM transient pressure-correction scheme), as implemented in `UEqn.H` and `pEqn.H`.

2.1.1 Modified pressure, hydrostatic potential and Boussinesq buoyancy

Rotating membrane devices exhibit very large hydrostatic pressure gradients, dominated by centrifugal effects. The physical pressure p may increase by several bars over a few centimetres of radius, while the dynamic variations induced by fluid motion are much smaller. If p is solved directly, the linear systems become ill–conditioned and the convergence of the PIMPLE algorithm deteriorates.

To avoid this, `centriMembSolver` follows the standard OpenFOAM strategy and works with three auxiliary fields:

- `p_rgh`: modified (dynamic) pressure

- `gh`: hydrostatic potential per unit mass
- `rhok`: dimensionless density ratio

The solver only solves for the dynamic component `p_rgh`; the hydrostatic contribution is treated analytically via `gh` and `rhok`.

Boussinesq density field Under the Boussinesq approximation, the density depends weakly on solute concentration,

$$\rho(\boldsymbol{x}) = \rho_0\, \rho_k(\boldsymbol{x}), \qquad |\rho_k - 1| \ll 1, \tag{3}$$

where the code field `rhok` corresponds to the scalar ρ_k. The Boussinesq approximation replaces ρ by the constant reference value ρ_0 in every term of the momentum equation *except* the body–force (buoyancy) term, where the actual density $\rho = \rho_0\rho_k$ is retained. This linearises the inertia, viscous and Coriolis terms while capturing the leading-order density-driven body force. Consistently with this approximation, the velocity field is treated as divergence–free,

$$\nabla \cdot \boldsymbol{U} = 0, \qquad \frac{\partial \rho}{\partial t} = 0, \tag{4}$$

but density variations ($\rho_k \neq 1$) still generate buoyancy forces.

Effective acceleration in the rotating frame Let $\boldsymbol{x}$ denote the position vector in the rotating frame and $\boldsymbol{x}_0$ a point on the rotation axis. Both are read from `constant/SRFProperties` (Single Reference Frame: the OpenFOAM dictionary that specifies the rotation axis through `origin` and `axis`, and the rotation rate through `rpm`). A fluid particle at rest in this frame is subject to gravity and centrifugal acceleration:

$$\boldsymbol{a}_{\text{eff}}(\boldsymbol{x}) = \boldsymbol{g} - \boldsymbol{\Omega} \times \big(\boldsymbol{\Omega} \times (\boldsymbol{x} - \boldsymbol{x}_0)\big), \tag{5}$$

where $\boldsymbol{g}$ is the gravitational acceleration and $\boldsymbol{\Omega}$ the angular velocity vector. Note that $-\boldsymbol{\Omega} \times (\boldsymbol{\Omega} \times \mathbf{r}_\perp) = +\Omega^2\, \mathbf{r}_\perp$, where $\mathbf{r}_\perp$ is the component of $\mathbf{r}$ perpendicular to the rotation axis: the centrifugal acceleration points radially outward.

The Coriolis term $-2\,\boldsymbol{\Omega} \times \boldsymbol{U}$ is non–conservative and is therefore *not* included in $\boldsymbol{a}_{\text{eff}}$.

Hydrostatic potential `gh` The key point is that the effective acceleration $\boldsymbol{a}_{\text{eff}}$ in (5) is conservative: it can be written as the gradient of a scalar potential per unit mass. OpenFOAM stores this potential in the field `gh`, defined so that

$$\nabla(\text{gh}) = \boldsymbol{a}_{\text{eff}}. \tag{6}$$

A convenient explicit expression is

$$\text{gh}(\boldsymbol{x}) = \boldsymbol{g} \cdot (\boldsymbol{x} - \boldsymbol{x}_{\text{ref}}) + \frac{1}{2}\,|\boldsymbol{\Omega} \times (\boldsymbol{x} - \boldsymbol{x}_0)|^2 - \text{gh}_{\text{ref}}, \tag{7}$$

where $\boldsymbol{x}_{\text{ref}}$ is a reference point and gh_{ref} is a constant offset. This offset does not affect the physics, since only gradients of gh enter the equations.

Physical meaning. gh is the hydrostatic potential per unit mass, combining gravitational and centrifugal contributions. Surfaces of constant gh correspond to equilibrium surfaces of a fluid at rest under gravity and rotation.

Why store the centrifugal potential in `gh`. Both gravity and the centrifugal acceleration of the rotating frame are conservative body forces and can therefore be absorbed into a single hydrostatic potential, with their density-fluctuation counterparts retained as the buoyancy drivers. This is the classical construction for rotating Boussinesq flows (Chandrasekhar 1961; Lopez, Marques & Avila 2013): explicitly, $-\rho_0\,\mathbf{\Omega}\times(\mathbf{\Omega}\times\mathbf{r}) = \nabla\big(\frac{1}{2}\rho_0|\mathbf{\Omega}\times\mathbf{r}|^2\big)$ is absorbed into the pressure gradient, leaving $-\rho'\,\mathbf{\Omega}\times(\mathbf{\Omega}\times\mathbf{r})$ to drive centrifugal buoyancy. In `centriMembSolver` we realise this within OpenFOAM's existing `gh`/`p_rgh` infrastructure (Eq. (7)); only the non-conservative Coriolis term remains as an explicit source in the momentum equation. The practical motivation is numerical conditioning: at high rotation rates the centrifugal hydrostatic pressure can dominate the absolute pressure field by orders of magnitude (~ 60 bar in high-throughput screening cartridges), and absorbing it into `p_rgh` keeps the pressure–velocity coupling well conditioned.

Modified pressure p_{rgh} In the solver, the physical pressure is reconstructed from the three fields `p_rgh`, `gh` and `rhok` as

$$p(\boldsymbol{x}) = \rho_0\big(p_{\mathrm{rgh}}(\boldsymbol{x}) + \rho_k(\boldsymbol{x})\,\mathrm{gh}(\boldsymbol{x})\big) + p_{\mathrm{ref}}, \tag{8}$$

where p_{rgh} corresponds to the OpenFOAM field `p_rgh` and p_{ref} is a constant reference pressure. Neglecting the constant p_{ref}/ρ_0, this can be rearranged as

$$p_{\mathrm{rgh}} = \frac{p}{\rho_0} - \rho_k\,\mathrm{gh}. \tag{9}$$

Thus p_{rgh} represents the *dynamic* part of the pressure: the large hydrostatic background is removed and only the smaller fluctuations are handled by the linear solver.

Momentum equation in p_{rgh} form Applying the Boussinesq approximation as stated above ($\rho \to \rho_0$ in inertia, viscous and Coriolis terms; $\rho = \rho_0\rho_k$ retained only in the body–force term), the incompressible rotating–frame Navier–Stokes equation becomes

$$\rho_0\left(\frac{\partial \boldsymbol{U}}{\partial t} + \boldsymbol{U}\cdot\nabla\boldsymbol{U} + 2\,\mathbf{\Omega}\times\boldsymbol{U}\right) = -\nabla p + \nabla\cdot\big(\mu_{\mathrm{eff}}\nabla\boldsymbol{U}\big) + \rho_0\rho_k\,\boldsymbol{a}_{\mathrm{eff}}. \tag{10}$$

Step-by-step transformation to the p_{rgh}-based form Using the definition (9),

$$p = \rho_0\,(p_{\mathrm{rgh}} + \rho_k\,\mathrm{gh})\,,$$

the pressure gradient becomes

$$\begin{aligned} -\nabla p &= -\rho_0\nabla(p_{\mathrm{rgh}} + \rho_k\,\mathrm{gh}) \\ &= -\rho_0\nabla p_{\mathrm{rgh}} - \rho_0\nabla(\rho_k\,\mathrm{gh}). \end{aligned} \tag{11}$$

Applying the product rule,

$$\nabla(\rho_k\,\mathrm{gh}) = \mathrm{gh}\,\nabla\rho_k + \rho_k\,\nabla(\mathrm{gh}) = \mathrm{gh}\,\nabla\rho_k + \rho_k\,\boldsymbol{a}_{\mathrm{eff}}, \tag{12}$$

where we used $\nabla(\mathrm{gh}) = \boldsymbol{a}_{\mathrm{eff}}$. Thus

$$-\nabla p = -\rho_0\nabla p_{\mathrm{rgh}} - \rho_0\,\mathrm{gh}\,\nabla\rho_k - \rho_0\rho_k\,\boldsymbol{a}_{\mathrm{eff}}. \tag{13}$$

Substituting (13) into (10) and cancelling the $\rho_0\rho_k\,\boldsymbol{a}_{\mathrm{eff}}$ terms on both sides yields

$$\rho_0\left(\frac{\partial \boldsymbol{U}}{\partial t} + \boldsymbol{U}\cdot\nabla\boldsymbol{U} + 2\,\mathbf{\Omega}\times\boldsymbol{U}\right) = -\rho_0\nabla p_{\mathrm{rgh}} - \rho_0\,\mathrm{gh}\,\nabla\rho_k + \nabla\cdot\big(\mu_{\mathrm{eff}}\nabla\boldsymbol{U}\big). \tag{14}$$

Since ρ_0 is constant, we can divide the entire equation by ρ_0 and introduce the effective kinematic viscosity $\nu_{\text{eff}} = \mu_{\text{eff}}/\rho_0$ to obtain the canonical form used in the solver:

$$\frac{\partial \boldsymbol{U}}{\partial t} + \boldsymbol{U} \cdot \nabla \boldsymbol{U} + 2\,\boldsymbol{\Omega} \times \boldsymbol{U} = -\nabla p_{\text{rgh}} - \text{gh}\,\nabla \rho_k + \nabla \cdot \big(\nu_{\text{eff}} \nabla \boldsymbol{U}\big). \tag{15}$$

Note that $\nu_{\text{eff}} = \nu(C_A) + \nu_t$ is retained *inside* the divergence operator because the molecular kinematic viscosity $\nu(C_A)$ is concentration–dependent (Eq. (8)).

The hydrostatic load has disappeared from the unknown pressure (p_{rgh}), and only the term proportional to $\nabla \rho_k$ remains as a Boussinesq buoyancy force.

Discrete buoyancy term in `centriMembSolver` The continuous term $-\,\text{gh}\,\nabla \rho_k$ is discretised in OpenFOAM using face–based gradients:

$$-\,\text{gh}\,\nabla \rho_k \quad \Longrightarrow \quad -\,\text{gh}_f\,\text{snGrad}(\rho_k), \tag{16}$$

where gh_f is the potential field interpolated to the faces, and $\text{snGrad}(\rho_k)$ is the surface–normal gradient of ρ_k. In the actual code this appears as

```
- ghf * fvc::snGrad(rhok)
```

inside the reconstruction of the right–hand side of the momentum equation.

Summary and physical interpretation

- gh is the hydrostatic potential per unit mass: it combines gravitational and centrifugal contributions and is computed from geometry, $\boldsymbol{g}$ and $\boldsymbol{\Omega}$.
- ρ_k (field `rhok`) encodes density variations relative to ρ_0 and is responsible for Boussinesq buoyancy.
- p_{rgh} (field `p_rgh`) is the pressure variable actually solved by PIMPLE; its gradients are much smaller than those of the physical pressure p.
- The buoyancy force appears as $-\,\text{gh}\,\nabla \rho_k$, discretised in the code as `- ghf*fvc::snGrad(rhok)`.
- The physical pressure used for membrane transport and post–processing is reconstructed via (8).

2.1.2 Turbulence modelling

Although centrifugal membrane filtration can operate in laminar, transitional or weakly turbulent regimes, `centriMembSolver` allows the user to select any turbulence model available in the `RASModel` hierarchy of OpenFOAM. The model is instantiated via the usual run-time selection mechanism in `createFields.H`:

$$\nu_{\text{eff}} = \nu + \nu_t(\mathbf{U}, k, \varepsilon, \dots). \tag{17}$$

Typical configurations rely on:

- direct laminar modelling (`simulationType laminar;`)
- one–equation eddy–viscosity models such as Spalart–Allmaras
- two–equation models such as standard or low-Re $k{-}\varepsilon$ and $k{-}\omega$

with parameters defined in `constant/turbulenceProperties`. Turbulent mass transport uses an eddy diffusivity $D_t = \nu_t/\text{Sc}_t$, where Sc_t is the turbulent Schmidt number.

2.1.3 Concentration-dependent viscosity

Many aqueous solute solutions (salts, sugars, cryoprotectants, cryoprotectant carriers) exhibit a strong dependence of viscosity on the solute concentration C_A. `centriMembSolver` incorporates a polynomial parameterisation:

$$\nu(C_A) = \nu_0 + \nu_1 C_A + \nu_2 C_A^2 + \cdots , \tag{18}$$

with coefficients ν_0, ν_1, ν_2 set in `transportProperties`. A clipping operator prevents unphysical values at high concentrations.

2.2 Solute and tracer transport

In the current release, the solver solves two advective–diffusive transport equations: one for a primary solute concentration C_A and one for a passive tracer concentration C_T. Both species are advected by the same velocity field $\mathbf{U}$ but may have different diffusion coefficients. The governing equations read

$$\frac{\partial C_A}{\partial t} + \nabla \cdot (C_A \mathbf{U}) = \nabla \cdot (D_{A,\text{eff}} \nabla C_A), \tag{19}$$

$$\frac{\partial C_T}{\partial t} + \nabla \cdot (C_T \mathbf{U}) = \nabla \cdot (D_{T,\text{eff}} \nabla C_T), \tag{20}$$

where $D_{A,\text{eff}} = D_{AB}(C_A) + D_t$ is the effective diffusivity of the solute including the concentration–dependent molecular diffusivity D_{AB} and a turbulent contribution D_t (if enabled), and $D_{T,\text{eff}} = D_{T,0} + D_t$ is the effective diffusivity of the tracer with $D_{T,0}$ constant and independent of concentration. Membrane boundary conditions for the solute and tracer are described in §3.3 and §3.4, respectively.

2.3 Coriolis treatment in PIMPLE

The Coriolis pseudo-force $\mathbf{f}_C = -2\,\mathbf{\Omega} \times \mathbf{U}$ is non-conservative and is therefore added explicitly to the momentum equation as a source term, evaluated with the velocity available at the current PIMPLE outer iteration $\mathbf{U}^{(k)}$:

$$\mathbf{f}_C^{(k)} = -2\,\mathbf{\Omega} \times \mathbf{U}^{(k)}. \tag{21}$$

The explicit treatment is appropriate at the small time steps imposed by the convective CFL constraint of DNS in rotating membrane systems and avoids the additional matrix structure that an implicit discretisation would require. For DNS runs the `momentumPredictor` is disabled to avoid spurious Courant excursions in strongly rotating flows; a single outer corrector per time step is normally sufficient under the CFL-bounded Δt. The corresponding code-level snippet is shown together with the rest of the momentum loop in §4.

3 Membrane Boundary Conditions

Membrane patches couple the feed region to the permeate side by imposing fluxes of solvent and solute according to physical models. `centriMembSolver` implements two specialised patch field classes: `membraneSolventFluxFvPatchVectorField` for the velocity and `membraneSoluteFluxFvPatchScalarFi` for the concentration. The feed side of the membrane is the patch corresponding to the field values; permeate side conditions are either fixed (for constant pressure/concentration) or inferred from the flux law.

3.1 Membrane physics

This subsection collects all the membrane-transport physics that underlies the implementations of §3.2–§3.4. It can be skipped on a first reading by users who only want dictionary syntax; the implementation subsections forward-reference equations defined here when needed.

Driving forces and osmotic pressure. Transmembrane transport is driven by hydrostatic $\Delta p = p_f - p_p$ and osmotic $\Delta\pi = \pi(C_f) - \pi(C_p)$ differences. The osmotic pressure is represented by a virial expansion in solute concentration,

$$\pi(C_A) = \sum_{k=1}^{N} a_k\, C_A^k, \tag{22}$$

with mass-based coefficients a_k of units $\mathrm{Pa\,m^{3k}\,kg^{-k}}$ (C_A in $\mathrm{kg\,m^{-3}}$). At infinite dilution the linear coefficient reduces to the van 't Hoff value $a_1 = i\,RT/M$, where i is the dissociation factor, R the gas constant, T absolute temperature, and M the solute molar mass; the higher-order a_k capture deviations from ideality.

Spiegler–Kedem solvent flux. The solvent (water) flux J_v [m/s] follows

$$J_v = A_h\big(\Delta p - \sigma\,\Delta\pi\big), \tag{23}$$

with hydraulic permeability A_h (dictionary key `Ah`) and reflection coefficient σ (dictionary key `sigma`).

Kedem–Katchalsky solute flux. The solute mass flux J_s [$\mathrm{kg\,(m^2\,s)^{-1}}$] is given by

$$J_s = (1-\sigma)\,J_v\,C_w + B\,(C_w - C_p), \tag{24}$$

where B is the intrinsic solute permeability (dictionary key `B`), C_w the wall concentration on the feed side, and C_p the permeate concentration. The intrinsic rejection coefficient is, by definition,

$$R_{\mathrm{int}} \equiv 1 - C_p/C_w. \tag{25}$$

Local mass balance at the membrane face. The membrane patch separates a feed-side bulk fluid (resolved by the DNS) from a permeate side. At the wall, the convective influx of solute, the diffusive removal back into the bulk feed, and the permeate outflux must balance:

$$J_v\,C_w \;-\; D_A\,\frac{\partial C_A}{\partial n}\bigg|_w \;=\; J_s \;=\; J_v\,C_p, \tag{26}$$

where $\partial/\partial n$ is the outward normal derivative on the feed-side patch (`snGrad` in OpenFOAM, defined as $(C_w - C_i)/\delta_n$ for the cell adjacent to the face, with δ_n the cell-centre–to–face distance). Equation (26) couples C_w to the resolved interior field; it makes *no* use of a phenomenological mass-transfer coefficient because the boundary layer is resolved by the DNS.

The *closure* required to solve (26) for C_w is provided differently by the two membrane-physics specialisations:

- **Intrinsic-rejection closure.** The membrane is assumed to reject a fixed fraction R_{int} of the solute regardless of operating point,

$$C_p = (1 - R_{\mathrm{int}})\,C_w. \tag{27}$$

Substituting (27) into (26) gives the Robin-type relation

$$D_A \left.\frac{\partial C_A}{\partial n}\right|_w = R_{\text{int}}\, J_v\, C_w. \tag{28}$$

- **Solute-permeability (Spiegler–Kedem) closure.** Combining the SK solute flux (24) with $J_s = J_v\, C_p$ and solving for C_p gives

$$C_p = \frac{B + (1-\sigma)\, J_v}{B + J_v}\, C_w \;\equiv\; \alpha(J_v; \sigma, B)\, C_w, \tag{29}$$

  so that the effective intrinsic rejection of an SK membrane is $R_{\text{int}}^{(\text{SK})}(J_v) = 1-\alpha = \sigma\, J_v/(B+J_v)$. Substituting (29) into (26) gives the corresponding Robin relation

$$D_A \left.\frac{\partial C_A}{\partial n}\right|_w = \frac{\sigma\, J_v^2}{B + J_v}\, C_w. \tag{30}$$

Implementation as a Robin (mixed) boundary condition. Both (28) and (30) have the canonical Robin form

$$k\,(C_w - C_i) \;=\; h_{\text{phys}}\, C_w, \qquad k \equiv D_A/\delta_n, \tag{31}$$

with $h_{\text{phys}} = R_{\text{int}}\, J_v$ (intrinsic-rejection closure) or $h_{\text{phys}} = \sigma\, J_v^2/(B + J_v)$ (solute-permeability closure). Solving for C_w,

$$C_w \;=\; \frac{k}{k - h_{\text{phys}}}\, C_i. \tag{32}$$

In OpenFOAM this is realised through the `mixedFvPatchScalarField` machinery, which evaluates the wall value as $C_w = \alpha_{\text{vF}}\, C_{\text{ref}} + (1-\alpha_{\text{vF}})\,(C_i + g\,\delta_n)$. With reference value and reference gradient set to zero, the implementation reproduces (32) by setting

$$\alpha_{\text{vF}} \;=\; -\frac{h_{\text{phys}}}{k - h_{\text{phys}}}, \tag{33}$$

which is *negative-valued by construction* under physical operating conditions ($0 < h_{\text{phys}} < k$). The OpenFOAM identifier `valueFraction` therefore stores an algebraic Robin weight rather than a fraction in $[0, 1]$, and negative values are correct.

Three modeling choices per species. Equations (23)–(24) contain four membrane parameters: A_h, σ, B, and the rejection (linked to σ and B via (25)). Different experimental scenarios fix different combinations of these parameters, and `centriMembSolver` therefore offers three modeling choices per species:

- Solvent flux (**U** BC, §3.2): the Spiegler–Kedem model fixes A_h and σ; the constant-flux model fixes J_v directly; the target-average-flux model tunes A_h on-the-fly to match a prescribed $\langle J_v \rangle$.
- Solute flux (C_A BC, §3.3): the intrinsic-rejection model fixes R_{int} and uses (28); the solute-permeability model fixes B (and optionally σ) and uses (30); the observed-rejection model tunes R_{int} on-the-fly to match a prescribed R_{obs}.
- Tracer flux (C_T BC, §3.4): same three options applied to a passive tracer with its own constant diffusivity D_T and parameters ($R_{T,\text{int}}$, B_T, σ_T).

3.2 Solvent flux boundary (U)

The solvent flux condition determines the normal component of **U** at the membrane. Three models are provided. In all three the local solvent flux is clipped to non-negative values, $J_v \leftarrow \max(0, J_v)$, so that the membrane never admits permeate-to-feed back-flow during transient or extreme conditions; this safeguard never affects steady-state results in the validated operating envelope but stabilises early-time behaviour while the dynamic pressure has not yet established.

Spiegler–Kedem model. Implements Eq. (23). Dictionary entries include the hydraulic permeability `Ah`, the reflection coefficient `sigma`, virial coefficients `virialCoeffs` for the osmotic pressure (with C_A in kg/m^3 and π in Pa), and an optional permeate-side pressure reference `pPermConst`. The example below sets `Ah = 0.5e-11`, `sigma = 1.0` and a virial expansion of the form $\pi(C_A) = a_1 C_A + a_2 C_A^2$:

```
membraneA
{
    type    membraneSolventFlux;
    model   SpieglerKedem;
    Ah      0.5e-11;        // [m/(Pa s)]
    sigma   1.0;            // [-]
    virialCoeffs (2.5e3 0); // a1 [Pa m3 kg-1], a2 [Pa m6 kg-2]
    pPermConst 0;           // [Pa]
    value  uniform (0 0 0);
}
```

Constant flux model. Prescribes a uniform solvent flux J_v independent of driving forces. Useful for benchmarking and when the membrane permeability is unknown. Example:

```
membraneA
{
    type    membraneSolventFlux;
    model   constantFlux;
    Jv      3.0e-5;        // [m/s]
    value   uniform (0 0 0);
}
```

Target average flux model. Tunes an effective permeability A_h^* such that the area-averaged flux matches a target `JvAverage`. Driving forces are still proportional to $\Delta p - \sigma \Delta \pi$ as in Spiegler–Kedem, but A_h^* is updated each time step within user-specified bounds `AhMin` and `AhMax`. Particularly useful for calibrating models against a measured average permeate flux. Example:

```
membraneA
{
    type        membraneSolventFlux;
    model       targetAverageFlux;
    JvAverage   3.0e-5;        // target <Jv> [m/s]
    AhMin       1.0e-13;       // lower bound on A_h^* [m/(Pa s)]
    AhMax       1.0e-9;        // upper bound on A_h^*
    sigma       1.0;
    virialCoeffs (7314.5 8.253);
```

```
    pPermConst  0;
    value       uniform (0 0 0);
}
```

Diagnostics. For each membrane patch the solver writes the instantaneous and time–windowed area–average of J_v. If window averaging is enabled (`windowAverage true`), the time scale `Twindow` sets the averaging duration. The internal field `Jv` is available for further post–processing.

3.3 Solute flux boundary (`CA`)

The solute boundary condition couples the solute transport to the solvent flux through the local mass balance derived in §3.1 (Eqs. (26) and following), and is implemented as an OpenFOAM `mixedFvPatchScalarField` (Robin BC). Three modeling closures are available.

Intrinsic rejection model. Fixes R_{int} directly; uses Eqs. (27) and (28). Dictionary entries: `model intrinsicRejection`, `Rint`, and the reference bulk concentration `CAb` used for the polarisation diagnostic Γ.

Solute permeability model. Fixes the solute permeability B (and optionally σ); uses Eqs. (29) and (30). Reduces to a solution–diffusion model when $\sigma = 0$. If `sigma` is specified it overrides the reflection coefficient from the solvent BC; otherwise σ is inherited from the solvent BC on `U`, defaulting to 1 if no membrane solvent BC is found. Dictionary entries: `model solutePermeability`, `B`, optionally `sigma`, and `CAb`.

Observed rejection model. Targets a prescribed observed rejection

$$R_{\text{obs}} \;=\; 1 - \frac{\langle J_v C_p \rangle}{\langle J_v \rangle\, C_b}, \tag{34}$$

where $\langle \cdot \rangle$ denotes an area average and C_b is the bulk reference concentration. The solver iteratively determines an intrinsic rejection R_{int} such that R_{obs} matches the target within a specified tolerance, applying Eq. (28) face-by-face with the inferred R_{int}. Dictionary entries: `model observedRejection`, `Robs` (target), `RintMin`, `RintMax`, `tol`, `maxIter`, and `CAb`. Useful when matching experimental rejection measurements.

Diagnostics. Per-face fluxes J_s, J_v and permeate concentration C_p are exposed via the patch fields `Js`, `Jv`, and `CAp`, and logged in `postProcessing/membraneSoluteFlux/<patch>/Rint_vs_time.dat` (columns: *time*, R_{int}, R_{obs}). The solver also writes the local concentration polarisation factor $\Gamma = (C_w - C_b)/C_b$ to the volume field `Gama`.

3.4 Tracer flux boundary (`CT`)

The tracer boundary condition reuses the same Robin BC machinery described in §3.1 (Eqs. (26)–(32)), applied to the passive tracer field C_T with diffusivity D_T and tracer-specific parameters $R_{T,\text{int}}$, B_T, σ_T. All operations on C_T are independent of those on C_A; the only coupling is through the shared solvent flux J_v. Three modeling closures are available, in direct analogy to the solute case.

Intrinsic rejection model. Tracer counterpart of (27) and (28): $C_{T,p} = (1 - R_{T,\text{int}})\, C_{T,w}$ and $D_T\, \partial C_T/\partial n|_w = R_{T,\text{int}}\, J_v\, C_{T,w}$. Dictionary entries: `model intrinsicRejection`, `RTint`, `CTb`.

Solute permeability model. Tracer counterpart of the SK closure (29)–(30) with (σ, B) replaced by (σ_T, B_T). When `sigmaT` is omitted, the default is $\sigma_T = 1$ (full rejection of the tracer); unlike the solute case, σ_T is *not* inherited from the solvent BC, since the reflection coefficient of a passive tracer is generally unrelated to that of the primary solute. Dictionary entries: `model solutePermeability`, `BT`, optionally `sigmaT`, `CTb`.

Observed rejection model. Tracer counterpart of (34), with (C_p, C_b) replaced by $(C_{T,p}, C_{T,b})$. The solver iteratively determines $R_{T,\text{int}}$ such that the achieved $R_{T,\text{obs}}$ matches a prescribed target. Dictionary entries: `model observedRejection`, `RTobs`, `RTintMin`, `RTintMax`, `tol`, `maxIter`, `CTb`.

Tracer dictionary example.

```
membraneB
{
    type     membraneTracerFlux;
    model    intrinsicRejection;
    RTint    1.0;          // perfectly rejected tracer
    CTb      1.0e-3;       // reference [kg/m^3]
    value    uniform 0;
}
```

Diagnostics. Per-face tracer flux $J_{s,T}$, solvent flux J_v and permeate tracer concentration $C_{T,p}$ are exposed via the patch fields `JsT`, `Jv` and `CTp`. The solver writes the observed tracer rejection and concentration polarisation factor $\Gamma_T = (C_{T,w} - C_{T,b})/C_{T,b}$ to the summary files for post-processing.

3.5 Membrane patch dictionaries

Tables 1 and 2 summarise the principal dictionary entries for solvent and solute boundary conditions. Most values default to physically reasonable numbers; only those shown as required must be provided.

4 Solver structure and implementation

The `centriMembSolver` follows a PIMPLE–type pressure–velocity coupling, extended with (i) concentration–dependent transport properties, (ii) a rotating reference frame, and (iii) custom membrane boundary conditions and diagnostics. The overall time loop is driven by `runTime.run()`, and at each time step a nested PIMPLE loop solves momentum, pressure and solute transport. Additional headers perform pressure reconstruction and conservation monitoring before the fields are written.

Table 1: Summary of membrane solvent flux boundary entries.

Key	Units	Description
`model`	–	`SpieglerKedem`, `constantFlux` or `targetAverageFlux`
`Ah`	m/(Pa s)	Hydraulic permeability for Spiegler–Kedem
`sigma`	–	Reflection coefficient σ; defaults to 1
`virialCoeffs`	$\mathrm{Pa\,m^{3k}\,kg^{-k}}$	List $(a_1, a_2, \dots)$ for osmotic pressure $\pi(C_A) = \sum_k a_k C_A^k$ (a_1 in $\mathrm{Pa\,m^3\,kg^{-1}}$, a_2 in $\mathrm{Pa\,m^6\,kg^{-2}}$, etc.)
`Jv`	m/s	Constant flux magnitude (constantFlux model)
`JvAverage`	m/s	Target area–average flux (targetAverageFlux)
`AhMin`, `AhMax`	m/(Pa s)	Limits on tuned permeability
`pPermConst`	Pa	Permeate-side reference pressure (default 0)
`windowAverage`, `Twindow`	–	Enable and set window for time–averaged output

Table 2: Summary of membrane solute flux boundary entries.

Key	Units	Description
`model`	–	`intrinsicRejection`, `solutePermeability` or `observedRejection`
`Rint`	–	Intrinsic rejection coefficient (intrinsicRejection)
`B`	m/s	Solute permeability (solutePermeability)
`sigma`	–	Optional reflection coefficient (overrides solvent BC; default 1)
`Robs`	–	Target observed rejection (observedRejection)
`CAb`	$\mathrm{kg/m^3}$	Reference bulk concentration for diagnostics
`RintMin`, `RintMax`	–	Bounds for inverse rejection search
`tol`, `maxIter`	–	Tolerance and maximum iterations for secant/bisection
`UName`	–	Name of velocity field (defaults to `U`)

4.1 Overall time loop and custom PIMPLE algorithm

For readers familiar with C++, the main structure of `centriMembSolver.C` can be summarised schematically as

```
createTime;
createMesh;
createFields;                // U, p_rgh, CA, Js, Jv, CAp, Gama,
                             // and tracer fields (CT, DT, JsT,
                             // CTp, GamaT) if solveTracer=true
createPimpleControl;         // pimpleControl
createRDeltaT;               // LTS auto-detection (sets bool LTS)
updateTransportProperties;   // nu(CA), DAB(CA), rhok

while (runTime.run())
{
 // LTS or CFL-based time-step control
 if (LTS) { #include "setRDeltaT.H" }
```

Table 3: Summary of membrane tracer flux boundary entries.

Key	Units	Description
`model`	–	`intrinsicRejection`, `solutePermeability` or `observedRejection`
`RTint`	–	Intrinsic tracer rejection coefficient (intrinsicRejection)
`BT`	m/s	Tracer permeability (solutePermeability)
`sigmaT`	–	Optional tracer reflection coefficient (default 1; full rejection)
`RTobs`	–	Target observed tracer rejection (observedRejection)
`CTb`	kg/m^3	Reference bulk tracer concentration for diagnostics
`RTintMin`, `RTintMax`	–	Bounds for inverse rejection search
`tol`, `maxIter`	–	Tolerance and maximum iterations for secant/bisection
`UName`	–	Name of velocity field (defaults to `U`)

```
else     { #include "CourantNo.H"; #include "setDeltaT.H" }

++runTime;
Info<< "Time = " << runTime.timeName() << endl;

while (pimple.loop())           // outer PIMPLE loop
{
 #include "UEqn.H"           // assemble & solve momentum
 #include "CAEqn.H"          // assemble & solve solute
 #include "updateTransportProperties.H"

 while (pimple.correct())    // inner pressure loop
 {
  #include "pEqn.H"
 }

 if (pimple.turbCorr())
 {
  turbulence->correct();
 }
}

#include "CTEqn.H"
#include "pReconstruct.H"
#include "monitorConservation.H"
#include "Gama.H"
#include "GamaT.H"

runTime.write();
runTime.printExecutionTime(Info);
}
```

Compared with a "standard" OpenFOAM transient solver:

- momentum (`UEqn.H`) and solute (`CAEqn.H`) are solved together at the beginning of each PIMPLE outer iteration;
- transport properties (viscosity, diffusivity, density) are updated *inside* the PIMPLE loop via `updateTransportProperties.H`, so they respond iteratively to the current concentration field;
- several post–processing steps (absolute pressure, conservation, membrane diagnostic Γ, body forces) are executed at the end of every time step.

This design makes the coupling between flow, solute transport and membrane behaviour explicit while still using the familiar PIMPLE machinery.

4.2 Field creation, membrane diagnostics and transport properties

Field creation is handled in two main headers:

- `createFields.H` reads the primary fields (`U`, `p_rgh`, `CA`, etc.) from the `0/` directory and the physical parameters from `constant/transportProperties` and `constant/SRFProperties`. It also constructs the turbulence model and creates all membrane diagnostic fields directly: `Js` (solute flux), `Jv` (solvent flux), `CAp` (permeate solute concentration) and `Gama` (dimensionless polarisation factor). When the user enables the tracer via the runtime switch `solveTracer` in `controlDict`, `createFields.H` additionally allocates the tracer fields `CT`, `DT`, `DTeff`, `JsT`, `CTp` and `GamaT` through `autoPtr` holders. All diagnostic fields are created with `AUTO_WRITE` so that they are automatically written at output times.

The header `updateTransportProperties.H` is called once before the time loop and then at each outer PIMPLE iteration. It performs three updates:

1. *Viscosity*: the laminar transport model `laminarTransport` is corrected using the current concentration field `CA`, so that the kinematic viscosity $\nu(\mathrm{CA})$ follows the chosen `CAViscosity` model.
2. *Molecular diffusivity*: the scalar field `DAB` is updated as a function of `CA`, using a polynomial relation. Boundary conditions on `DAB` are corrected afterwards.
3. *Density*: a non–dimensional density field `rhok` is recomputed according to a Boussinesq–type law, $\rho_k = 1 + \beta\,\mathrm{CA}$, and its boundary conditions are corrected.

Updating these properties inside the PIMPLE loop strengthens the coupling between momentum and solute transport: any change in `CA` is immediately reflected in ν, D_{AB} and ρ_k.

4.3 Momentum and pressure equations

Inside the outer `while (pimple.loop())`:

- `UEqn.H` assembles the transient momentum equation using implicit time discretisation (`fvm::ddt`), a convection term `fvm::div(phi,U)`, viscous/turbulent stresses via `turbulence->divDevReff` and a Coriolis source term $2\rho\,\mathbf{\Omega} \times \mathbf{U}$ for the rotating frame. The equation is then solved for the velocity field `U`.

- `CAEqn.H` assembles and solves the unsteady advection–diffusion equation for the solute concentration `CA`. The temporal and spatial discretisations follow the user-specified schemes in `system/fvSchemes`; for the tutorials a bounded convection scheme is recommended together with the time scheme appropriate for the chosen ddt (`Euler`, `CrankNicolson`, or `localEuler` when LTS is enabled).

After these solves, `updateTransportProperties.H` is called again to refresh $\nu(\mathrm{CA})$, $D_{AB}(\mathrm{CA})$ and ρ_k based on the newly updated `CA` field.

The pressure equation is then handled in the inner loop `while (pimple.correct())` through `pEqn.H`. This step enforces continuity, updates `p_rgh` (and the associated face flux `phi`) and accounts for the rotation–induced hydrostatic term and the specified pressure reference. When `pimple.turbCorr()` is true, the turbulence model is also corrected in the same outer iteration.

4.4 Membrane boundary conditions and the Γ diagnostic

The actual membrane physics is implemented in custom boundary condition classes applied to `U` and `CA`. These boundary conditions:

- compute, for each membrane face, the solvent flux J_v and solute flux J_s according to the selected transport model (e.g. pressure–driven with osmotic correction);
- update the permeate concentration C_p and a dimensionless diagnostic Γ that characterises local solute rejection or concentration polarisation.

At volume level, these per–face quantities are exported into the membrane diagnostic fields created in `createFields.H` (`Js`, `Jv`, `CAp`, `Gama`, and the optional tracer fields when `solveTracer=true`). The header `Gama.H` is responsible for copying the per–face Γ values from the membrane solute boundary condition into the boundary of the volume field `Gama`, so that `Gama` can be visualised and sampled like any other `volScalarField`. For efficiency, `Gama.H` caches the membrane patch indices and a pointer to the `CA` field the first time it is called.

4.5 Post-processing: pressure reconstruction, conservation and diagnostics

After the PIMPLE loop converges at a given time, the following post–processing headers are executed before writing:

`pReconstruct.H` The solver works internally with the modified pressure `p_rgh` to simplify the treatment of gravity in a rotating frame. For analysis, it is convenient to have the absolute pressure field. The header `pReconstruct.H` reconstructs

$$p = (p_\mathrm{rgh} + \rho_k\, g\, h)\, \rho_0 + p_{\mathrm{ref}},$$

where ρ_0 is a reference density and p_{ref} the chosen pressure reference, and then calls `p.correctBoundaryConditio` to keep the boundaries consistent.

`monitorConservation.H` This header checks global solute conservation in a parallel–safe way. It requires the fields `CA`, `U`, `Js`, `Jv`, `CAp` and the face flux `phi`; an optional `Deff` field can be supplied to account for diffusive fluxes at walls. It builds (or appends to) the file

`postProcessing/conservation/cons.dat`,

writing at each output time the total solute mass in the domain, the time derivative of that mass, the integrated inlet, outlet, membrane and diffusive contributions, and a residual and relative error that measure how well the discrete balance is satisfied. This file is the main tool to verify that the coupled flow–membrane scheme is globally conservative.

`Gama.H` and `GamaT.H` As described above, `Gama.H` maps the per–face diagnostic Γ from the membrane solute boundary condition into the volume field `Gama`. In the 2026 release, a companion header `GamaT.H` performs the analogous mapping for the tracer polarisation factor `GamaT`. This is done patch by patch on the membrane boundaries, with simple consistency checks (same number of faces) to avoid copying errors.

At the end of this post–processing stage, `runTime.write()` writes all `AUTO_WRITE` fields (`U`, `p`, `p_rgh`, `CA`, `Js`, `Jv`, `CAp`, `Gama`, and the tracer counterparts when enabled), and the solver advances to the next time level.

5 Case studies with `centriMembSolver`

This section presents an example case that illustrates how to use `centriMembSolver`. It is designed to be readable even for readers who are not familiar with OpenFOAM.

In OpenFOAM, each simulation ("case") is stored in a directory with a standard structure:

- `0/` contains the *initial and boundary conditions* for all fields (for example velocity, pressure and concentration);
- `constant/` contains the *physical properties* (for example fluid properties, solute properties, membrane parameters, rotation settings);
- `system/` contains the *numerical setup* (mesh generation with `blockMesh`, time step controls, linear solvers, etc.).

To run a case, one typically:

1. generates the mesh (for example with `blockMesh` or a helper script);
2. runs the solver (here `centriMembSolver`);
3. inspects the results in ParaView or in text files written in `postProcessing/`.

Below we describe a simple 2D slit case (`rotatingSlit`) that exercises the full pipeline (mesh, BCs, solver, post-processing).

5.1 `rotatingSlit`

The example case `rotatingSlit` distributed with the solver provides a simple yet instructive demonstration. It consists of a two-dimensional slit domain rotated about its centre. The membrane occupies a section of the side wall; the remaining boundaries are inlet, outlet, symmetry and empty patches.

Geometry and mesh

The geometry is defined by `system/blockMeshDict`. A narrow slit of length 4 cm and width 1 mm is discretised into a structured mesh. One boundary, named `membraneB`, is assigned the membrane boundary conditions. A second boundary `membraneA` is present but disabled in this case. Front and back patches are declared as `empty` to produce a two-dimensional simulation.

Initial and boundary conditions

The inlet velocity is fixed at $\mathbf{U}_{\text{in}} = (0.01, 0, 0)$ m/s along the slit axis. The initial solute concentration is uniform ($5\,\text{kg/m}^3$) and the inlet condition sets `CAin` to a higher or lower value to induce concentration polarisation. The membrane on patch `membraneB` is configured with

- a target average solvent flux of 3×10^{-5} m/s,
- limits `AhMin = 1e-13` and `AhMax = 1e-9`,
- reflection coefficient $\sigma = 1$, and
- virial coefficients $(7314.5\,\text{Pa}\,\text{m}^3\,\text{kg}^{-1}, 8.253\,\text{Pa}\,\text{m}^6\,\text{kg}^{-2})$.

On the solute side the membrane uses the intrinsic rejection model with `Rint = 0.99` and reference concentration `CAb = 1.0`. A fixed–flux pressure condition ensures continuity of volumetric flow.

Running the case

To run the example, follow these steps in the case directory:

1. Compile the solver (only needed once) by executing `wmake` in the `solver` directory.
2. Generate the mesh using

   ```
   blockMesh
   ```

3. Adjust `constant/SRFProperties` to set the rotation speed $\mathbf{\Omega}$ if desired. The default example uses `rpm = 0`, but any non–zero value can be specified.
4. Run the simulation with

   ```
   centriMembSolver -case rotatingSlit > log &
   ```

5. Monitor progress via the log or the files in `postProcessing`. Key outputs include `Rint_vs_time.dat` (intrinsic rejection over time), `calibratedProperties.dat` (effective permeability for target average flux) and `cons.dat` (mass conservation).

Figure 1 shows a real run of the bundled `rotatingSlit` tutorial. Two transients are visible.

Panel (a) — targetAverageFlux calibration. The `targetAverageFlux` solvent model dynamically adjusts the effective hydraulic permeability A_h^* at every time step so that the area-averaged solvent flux matches the prescribed `JvAverage`. Starting from a low initial value (no polarisation, weak osmotic counter-pressure), A_h^* converges to its steady operating value (here

4.36×10^{-11} m (Pa s)$^{-1}$) within ~15 s of simulated time. By construction the model keeps $\langle J_v \rangle$ exactly at the target throughout.

Panel (b) — concentration polarisation signature. The intrinsic rejection imposed at the membrane is constant by construction ($R_{\text{int}} = 0.99$, dashed line), while the observed rejection

$$R_{\text{obs}}(t) = 1 - \frac{\langle J_v C_p \rangle(t)}{\langle J_v \rangle(t)\, C_b}$$

descends from values close to R_{int} at the onset of permeation (no polarisation yet) towards a steady value substantially below R_{int} ($R_{\text{obs}} \approx 0.60$ here), reflecting the equilibrium polarisation that the simulation establishes at the membrane. The $R_{\text{int}} - R_{\text{obs}}$ gap is the operational signature of concentration polarisation, and is automatically logged under `postProcessing/membraneSoluteFlux/<patch>` (columns: time, R_{int}, R_{obs}).

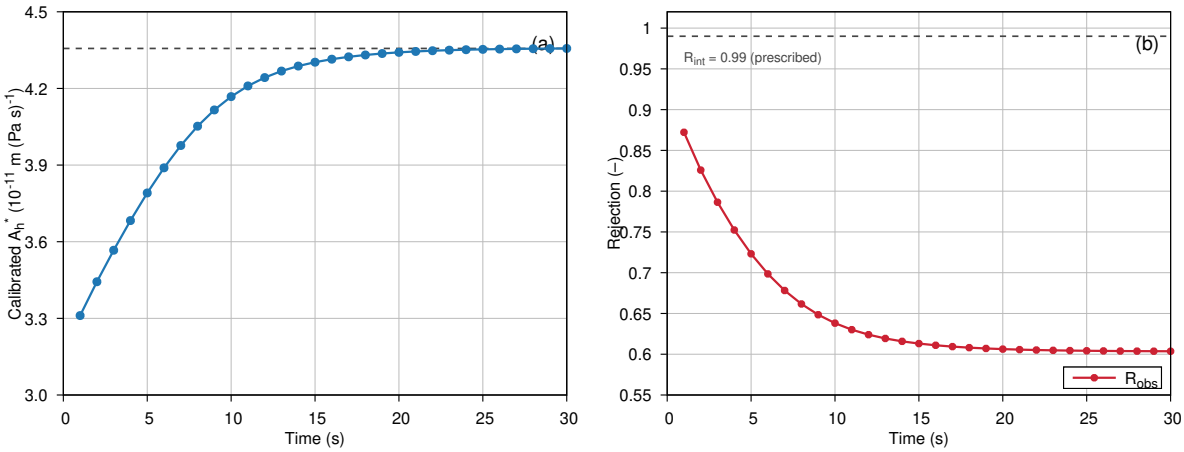

Figure 1: Real outputs of the bundled `rotatingSlit` tutorial (2D slit, $L \times h = 4\,\text{cm} \times 1\,\text{mm}$, NaCl-like feed at $5\,\text{kg/m}^3$, $\sigma = 1$, $R_{\text{int}} = 0.99$, `JvAverage`$= 3 \times 10^{-5}$ m/s). (a) Calibrated hydraulic permeability $A_h^*(t)$ extracted from the patch dictionary at each write time, showing the `targetAverageFlux` model converging to the steady operating permeability ($\sim 4.36 \times 10^{-11}$ m (Pa s)$^{-1}$). (b) Time evolution of the observed rejection R_{obs} on patch `membraneB` (red), starting near R_{int} (dashed line, prescribed) and stabilising around 0.60 as the polarised layer builds up. Data from `postProcessing/membraneSoluteFlux/membraneB/Rint_vs_time.dat` and the saved patch dictionaries (`<time>/U`).

6 Practical guidance

Stability and time stepping. With the Coriolis source treated explicitly, the convective Courant condition is the dominant time-step constraint; keep the Courant number below ~ 1 near the membrane. When using `targetAverageFlux`, choose the bounds `AhMin` and `AhMax` narrowly around the expected steady operating permeability: a range that is too wide allows the calibrated A_h^* to wander outside the physical regime and may produce oscillatory behaviour. When using `observedRejection`, pick a tight tolerance `tol` and verify that the target R_{obs} lies inside the physically realistic interval $[\,R_{\text{int,min}},\, R_{\text{int,max}}\,]$ – otherwise the inverse search hits the bracket and reports a clamped value rather than the target.

Parallel consistency. All membrane boundary conditions and diagnostics use parallel reductions (`Pstream::reduce`, `returnReduce`) and are safe out of the box on decomposed meshes. For

complex geometries or very fine meshes, however, it is advisable to invoke `redistributePar` after `decomposePar` so that each membrane patch is owned by a single processor: this avoids subtle load-balancing artefacts in the area-weighted averages. The conservation monitor (`postProcessing/conservati` provides a quick sanity check on mass balances across processor boundaries.

7 References

1. Spiegler, K. S. & Kedem, O. (1966). Thermodynamics of hyperfiltration (reverse osmosis): criteria for efficient membranes. *Desalination*, 1, 311–326.

2. Wu, J. J. (2019). On the application of the Spiegler–Kedem model to forward osmosis. *BMC Chemical Engineering*, 1, article 15. doi:10.1186/s42480-019-0014-8. (Discusses how the S–K model accommodates deviations from ideal semi-permeability.)

3. Chandrasekhar, S. (1961). *Hydrodynamic and Hydromagnetic Stability*. Clarendon Press, Oxford. International Series of Monographs on Physics. (Classical reference for absorbing the centrifugal term into the pressure gradient in rotating Boussinesq flows.)

4. Lopez, J. M., Marques, F. & Avila, M. (2013). The Boussinesq approximation in rapidly rotating flows. *Journal of Fluid Mechanics*, 737, 56–77. doi:10.1017/jfm.2013.558. (Explicit modern formulation of the gravity + centrifugal absorption; eqs. (2.1)–(2.3) used as reference for the `gh`/`p_rgh` construction in this solver.)